

Scope Document — VIVE Reconciliation (Final Architecture)

Prepared for:

Engineering Managers, Solution Architects, Business Stakeholders

Project Type:

Internal AI-assisted vendor statement reconciliation tool — final target architecture, including confirmed multi-user requirement

1. Project Overview

VIVE Reconciliation is a Python-based reconciliation tool built for VIVE Collision, a multi-shop auto body repair company (~79 shops, primarily Northeast US). Vendor suppliers submit monthly PDF statements; the AP team historically cross-checked these manually against ERP records.

This document reflects the final agreed architecture after adopting selected patterns from a review of the Scrutin (AI Invoice Reviewer) codebase, updated to account for a confirmed requirement: 5-10 people will use the system, with mixed often/occasional usage — not a single operator as originally scoped.

2. Scope (In Scope)

- **Core functionality:** Ingestion of vendor statement PDFs in any vendor-specific layout, without per-vendor configuration.
- **AI extraction:** Claude (Haiku 4.5, primary) with fallback to pdfplumber + Tesseract OCR (non-AI, free) for scanned documents if Claude is unavailable.
- **AI model decision:** Claude (Haiku 4.5) is the extraction engine — already available with no new setup required, and comfortably handles VIVE's PDF volume. A single AI provider keeps the system simple to maintain (one API key, one billing dashboard) at VIVE's scale. pdfplumber+OCR remains as a free, non-AI last resort, low-maintenance to keep now that Docker's containerization removes setup complexity. Claude also remains the model used for the optional --explain narrative step. Which method actually processed a given document is already recorded (document_intake_log.extraction_method, ai_audit_log.ai_provider) — no new tracking needed.
- **Processing pipeline:** Same four-stage pipeline (intake/extraction, mock ERP generation, deterministic matching, reporting), containerized via Docker for reproducible execution across machines.
- **User interface:** A complete Streamlit dashboard: uploading a vendor PDF directly triggers the existing pipeline (extraction → matching → reporting) — not a read-only viewer. Also accessible via the existing command line.
- **Multi-user infrastructure:** A real shared database (Azure SQL) replacing SQLite, a background job queue so uploads don't block each other, per-user logins (so the disposition record shows who resolved what), and shared hosting (Azure App Service) so all 5-10 users reach the same instance. The queue is a table (queued/processing/done/failed per job) with automatic re-queuing if a worker crashes mid-job, and reuses the existing document-hash check so the same file is never processed twice.
- **Document retention:** Original vendor PDFs retained in Azure Blob Storage, keyed by vendor name + year/month + the same SHA-256 hash already used for extraction caching — preventing duplicate storage of the same file. Upload metadata (who, when, which vendor) is stored alongside each file, and a new column on document_intake_log links each processed statement to its stored file for later retrieval.
- **Exception disposition:** Every resolved exception recorded with who resolved it, when, and how — suppressing repeat flags for the exact same previously-resolved exception on reprocessing.
- **Schema management:** Every database structure change tracked as a numbered, versioned script, recorded in a schema_version table.
- **Validation/reconciliation logic:** Same two-level deterministic matching hierarchy (Level 1 exact invoice match, Level 3 RO+amount fallback), no AI component, with explicit tagging for rows missing required fields (distinct from a genuine non-match) rather than silently failing or guessing.

- **Configuration:** Confidence cutoffs and provider-specific trim limits consolidated into config, alongside the existing `matching_rules.json`.
- **Design documentation:** A Rules doc cataloguing deliberate design decisions already made (e.g., no invoice-suffix stripping, cache hit requires `row_count > 0`) to prevent accidental regression.

3. Out of Scope

- The system does not approve, reject, or execute payments — it identifies discrepancies for human review.
- Formats other than PDF are not supported.
- Live NetSuite ERP integration is not implemented; ERP comparison still uses a generated mock dataset.
- Vendor-specific configuration or onboarding remains intentionally unsupported.
- Semantic/AI-driven judgment inside the matching decision is not introduced — matching remains 100% deterministic.
- Full role-based permissions (e.g. distinct Admin vs. Reviewer capabilities) are not implemented — per-user logins identify who did what, but everyone has the same access level, since everyone using the dashboard does the same job today.
- A document-level confidence gate (holding an entire statement if any part of it is low-confidence, rather than only the affected row) is not implemented in this phase — deferred until a specific, observed case shows the current per-row handling is letting an error through.
- An AI-driven “does this look odd” advisory check on individual line items is not implemented — deferred pending evidence that deterministic matching alone is missing real issues, and pending a data gap (no current vendor-type/category field to compare a line item's description against).

4. Assumptions

- Input documents are legible PDFs (text-based or scanned); non-PDF formats are out of scope.
- ERP reference data remains available via the mock ERP generator in the same normalized schema.
- 5-10 people will use the dashboard, with a mix of frequent and occasional use — not a fixed, predictable schedule.
- A human (AP/operations analyst or another authorized user) reviews the exception report before any corrective or payment action, and records a disposition for each resolved exception.
- The system runs inside a Docker container, deployed on shared hosting reachable by all users — not a single operator's laptop.
- Azure SQL Database, Azure Blob Storage, and Azure App Service are available and budgeted for.

5. Deliverables

- Reconciliation report summarizing matched and unmatched invoices per run, annotated with disposition history.
- Exception report distinguishing genuinely unmatched rows from rows that could not be checked due to missing fields.
- Optional AI-generated explanatory narrative per exception (`--explain` flag).
- Streamlit dashboard: PDF upload that directly triggers the pipeline, run history, exception list with disposition status, and a form to record a new disposition.
- Per-user login system for the dashboard.
- Background job queue so multiple uploads process without blocking each other.
- AI audit log recording every extraction call, supplemented by a disposition/audit table recording every human resolution (who, when).
- Dockerfile and setup instructions enabling one-command environment reproduction.
- Rules doc cataloguing deliberate design decisions.
- Versioned schema migration scripts and a `schema_version` tracking table.

6. Stakeholders / Users

- Primary users: 5-10 VIVE AP/operations staff executing reconciliation runs and recording exception dispositions, via CLI or dashboard.
- Business users: VIVE operations leadership reviewing exception outcomes and resolution history.
- Technical users: DataGrokr engineering team maintaining and extending the pipeline.
- Client: VIVE Collision.

7. Constraints

- New recurring infrastructure cost, where there was none before: approximately \$5/month for the database (Azure SQL, Basic tier) and approximately \$13/month for hosting (Azure App Service, Basic tier) — roughly \$18-20/month combined, plus a negligible few cents/month for Blob storage.
- ERP comparison continues to rely on a mock dataset pending live ERP integration.
- Extraction accuracy remains dependent on Claude's availability — the pdfplumber+OCR fallback covers loss of AI access but only for the extraction quality a non-AI table parser can achieve.
- Disposition recording depends on users actually logging resolutions — the benefit (suppressing repeat flags) only materializes if the workflow is actually used.
- Per-user logins identify who did what but do not restrict what any given user can do — there is no separate admin/reviewer permission tier.

Scope Comparison — Original Single-Operator Scope vs. Final Architecture

The following summarizes what changed between the original single-operator scope and the final architecture reflecting the confirmed multi-user requirement.

Dimension	Original Scope	Final Architecture
Users	Single operator	5-10 people, mixed often/occasional use
Interface	Command-line only	CLI still works; complete Streamlit dashboard (upload triggers the pipeline directly)
Database	SQLite, single file	Azure SQL Database (Basic tier) — supports concurrent multi-user access
Processing	Synchronous, one PDF at a time	Background job queue — uploads don't block each other
Access control	None	Per-user logins — identifies who resolved what, no separate permission tiers
Hosting	Local laptop	Azure App Service — shared, always-on instance reachable by all users
Document retention	PDFs read once locally, not retained	Retained permanently in Azure Blob Storage, hash-keyed with metadata + a database link
Exception memory	Every run re-flags all exceptions fresh	Resolved exceptions suppressed/annotated on exact repeat
Schema management	Manual edits, no version history	Versioned migrations tracked in schema_version
Design-decision record	Scattered in code comments	Central Rules doc with references at each enforcement point
Missing-field handling	Could fail matching silently or ambiguously	Explicitly tagged “not checked”, distinct from a genuine non-match
Truncated AI response handling	Salvage attempted first	Truncation detected explicitly, falls back immediately, salvage as backup
Threshold configuration	Some hardcoded in source	Consolidated into config
Matching logic	2-level deterministic hierarchy, no AI	Unchanged — same logic, same guarantee of no AI involvement
Recurring cost	\$0	~\$18-20/month (database + hosting), plus negligible Blob storage cost
Confidence gate (document-level)	Not implemented	Still not implemented — deferred until a real triggering case is observed